

## OD LE 0

### NI FILE APIs

#### General File APIs:

Files in a NI and POSIX system may be any one of the following types:

- Regular file.
- Directory File.
- FIFO file.
- Block device file.
- Character device file.
- Symbolic file.

There are special API's to create these types of files. There is a set of Generic API's that can be used to manipulate and create more than one type of files. These API's are:

#### ❖ open:

- ✓ This is used to establish a connection between a process and a file i.e. it is used to open an existing file for data transfer function or else it may be also be used to create a new file.
- ✓ The returned value of the open system call is the file descriptor (row number of the file table), which contains the inode information.
- ✓ The prototype of open function is:

```
include<sys/types.h>
include<sys/fcntl.h>

int open(const char  pathname, int accessmode, mode_t permission);
```

- ✓ If successful, open returns a nonnegative integer representing the open file descriptor.
- ✓ If unsuccessful, open returns -1.
- ✓ The first argument is the name of the file to be created or opened. This may be an absolute pathname or relative pathname.
- ✓ If the given pathname is symbolic link, the open function will resolve the symbolic link reference to a non symbolic link file to which it refers.
- ✓ The second argument is access modes, which is an integer value that specifies how actually the file should be accessed by the calling process.
- ✓ Generally the access modes are specified in . Various access modes are:
  - O\_RDONLY → Open for reading file only.
  - O\_WRONLY → Open for writing file only.
  - O\_RDWR → Opens for reading Writing file.

- ✓ There are other access modes, which are termed as access modifier flags, and one or more of the following can be specified by bitwise ORing them with one of the above access mode flags to alter the access mechanism of the file.
  - O\_APPEND → Append data to the end of file.
  - O\_CREAT → Create the file if doesn't exist.
  - O\_EXCL → Generate an error if O\_CREAT is also specified the file already exists.
  - O\_TRUNC → If file exists discard the file content set the file size to zero bytes.
  - O\_NONBLOCK → Specify subsequent read or write on the file should be non blocking.
  - O\_NOCTTY → Specify not to use terminal device file as the calling process control terminal.
- ✓ To illustrate the use of the above flags, the following example statement opens a file called /usr/SRP/usp for read and write in append mode: `int fd=open("/usr/SRP/usp",O_RDWR | O_APPEND,0);`
- ✓ If the file is opened in read only, then no other modifier flags can be used.
- ✓ If a file is opened in write only or read write, then we are allowed to use any modifier flags along with them.
- ✓ The third argument is used only when a new file is being created. The symbolic names for file permission are given in the table in the previous page.

| Symbol  | Meaning                        |
|---------|--------------------------------|
| S_IRUSR | Read by owner                  |
| S_IWUSR | Write by owner                 |
| S_IXUSR | Execute by owner               |
| S_IRWXU | Read, write, execute by owner  |
| S_IRGRP | Read by group                  |
| S_IWGRP | Write by group                 |
| S_IXGRP | Execute by group               |
| S_IRWXG | Read, write, execute by group  |
| S_IROTH | Read by others                 |
| S_IWOTH | Write by others                |
| S_IXOTH | Execute by others              |
| S_IRWXO | Read, write, execute by others |

❖ creat:

- ✓ This system call is used to create new regular files.
- ✓ The prototype of creat is

```
include <sys/types.h>

include<unistd.h>

int creat(const char  pathname, mode_t mode);
```

- ✓ Returns: file descriptor opened for write only if OK, on error.
- ✓ The first argument pathname specifies name of the file to be created.
- ✓ The second argument mode\_t, specifies permission of a file to be accessed by owner group and others.
- ✓ The creat function can be implemented using open function as:
 

```
define creat(path_name, mode)
open (pathname, O_WRONLY O_CREAT O_TRUNC, mode);
```

**Program:**

```
define _POSIX_SOURCE
include<fcntl.h>
include<sys/types.h>
include<sys/stat.h>
include<unistd.h>
undef _POSIX_SOURCE
include<stdio.h>
include<string.h>

int main()

char fn[] = "Nirmal", text[] = "Nirmal is a student of 5th sem CSE ";
int fd;

if ((fd = creat(fn, S_IRWXU | S_IRWXG | S_IRWXO)) < 0)
    perror("creat() error");
else

    write(fd, text, strlen(text));
    close(fd);

return(fd);
```

❖ read:

- ✓ The read function fetches a fixed size of block of data from a file referenced by a given file descriptor.
- ✓ The prototype of read function is:

```
include <sys/types.h
include <unistd.h
size_t read(int fdesc, void buf, size_t nbytes);
```

- ✓ If successful, read returns the number of bytes actually read.
- ✓ If unsuccessful, read returns -1.
- ✓ The first argument is an integer, fdesc that refers to an opened file.
- ✓ The second argument, buf is the address of a buffer holding any data read.
- ✓ The third argument specifies how many bytes of data are to be read from the file.
- ✓ The size\_t data type is defined in the header and should be the same as unsigned int.
- ✓ There are several cases in which the number of bytes actually read is less than the amount requested:
  - When reading from a regular file, if the end of file is reached before the requested number of bytes has been read. For example, if 10 bytes remain until the end of file and we try to read 20 bytes, read returns 10. The next time we call read, it will return 0 (end of file).
  - When reading from a terminal device. Normally, up to one line is read at a time.
  - When reading from a network. Buffering within the network may cause less than the requested amount to be returned.
  - When reading from a pipe or FIFO. If the pipe contains fewer bytes than requested, read will return only what is available.

**Program:**

```
include <sys/types.h
include <unistd.h
include <fcntl.h
include <stdio.h

int main()

    int fd,nob;
    char c[ 5 ];

    fd=open( "s",O_RDONLY);
    nob=read(fd,c,sizeof(c));
```

```

if(nob == 0)
    printf( "%s", c);
else
    perror( "Reading Error ");

close(fd);
return 0;

```

### ❖ write:

- ✓ The write system call is used to write data into a file.
- ✓ The write function puts data to a file in the form of fixed block size referred by a given file descriptor.
- ✓ The prototype of write is

```

include <sys/types.h>
include <unistd.h>
size_t write(int fd, const void *buf, size_t size);

```

- ✓ If successful, write returns the number of bytes actually written.
- ✓ If unsuccessful, write returns -1.
- ✓ The first argument, fd, is an integer that refers to an opened file.
- ✓ The second argument, buf, is the address of a buffer that contains data to be written.
- ✓ The third argument, size, specifies how many bytes of data are in the buf argument.
- ✓ The return value is usually equal to the number of bytes of data successfully written to a file. (size value).

### Program:

```

include <sys/types.h>
include <fcntl.h>
include <stdio.h>
include <string.h>
include <unistd.h>

int main()

{
    int fd, nob;
    char c[] = "This is sample program to write ";
    fd = open( "file.txt", O_WRONLY, 0 );
    nob = write(fd, c, strlen(c));
}

```

```

if(nob = )
    printf( Successfully written to the file );
else
    perror( write Error );
close(fd);
return 0;

```

#### ❖ close:

- ✓ The close system call is used to terminate the connection to a file from a process.
- ✓ The prototype of the close:

```

include<unistd.h
int close(int fdesc);

```

- ✓ If successful, close returns 0.
- ✓ If unsuccessful, close returns -1.
- ✓ The argument fdesc refers to an opened file.
- ✓ Close function frees the unused file descriptors so that they can be reused to reference other files. This is important because a process may open up to OPEN\_MAX files at any time and the close function allows a process to reuse file descriptors to access more than OPEN\_MAX files in the course of its execution.
- ✓ The close function de allocates system resources like file table entry and memory buffer allocated to hold the read/write.

#### ❖ fcntl:

- ✓ The fcntl function helps a user to query or set flags and the close on exec flag of any file descriptor.
- ✓ The prototype of fcntl is:

```

include<fcntl.h
int fcntl(int fdesc, int cmd, .....);

```

- ✓ The first argument is the file descriptor.
- ✓ The second argument cmd specifies what operation has to be performed.
- ✓ The third argument is dependent on the actual cmd value.
- ✓ The possible cmd values are defined in header.

| Cmd value | Use                                                                                                                                                                                                                                                                         |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F_GETFL   | Returns the access control flags of a file descriptor fdesc.                                                                                                                                                                                                                |
| F_SETFL   | Sets or clears access control flags that are specified in the third argument to fcntl. The allowed access control flags are O_APPEND & O_NONBLOCK.                                                                                                                          |
| F_GETFD   | Returns the close-on-exec flag of a file referenced by fdesc. If a return value is zero, the flag is off; otherwise on.                                                                                                                                                     |
| F_SETFD   | Sets or clears the close-on-exec flag of a desc. The third argument to fcntl is an integer value, which is 0 to clear the flag, or 1 to set the flag.                                                                                                                       |
| F_DUPFD   | Duplicates file descriptor fdesc with another file descriptor. The third argument to fcntl is an integer value which specifies that the duplicated the descriptor must be greater than or equal to that value. The return value of fcntl is the duplicated file descriptor. |

- ✓ The fcntl function is useful in changing the access control flag of a file descriptor.
- ✓ For example: after a file is opened for blocking read write access and the process needs to change the access to non blocking and in write append mode, it can call:

```
int cur_flags=fcntl(fdesc,F_GETFL);
int rc=fcntl(fdesc,F_SETFL,cur_flag | O_APPEND | O_NONBLOCK);
```

- ✓ The following example reports the close on exec flag of fdesc, sets it to on afterwards:

```
cout<<fdesc<<"close on exec"<<fcntl(fdesc,F_GETFD)<<endl;
(void)fcntl(fdesc,F_SETFD, 1); //turn on close on exec flag
```

- ✓ The following statements change the standard input of a process to a file called FOO:

```
int fdesc=open("FOO",O_RDONLY);           //open FOO for read
close(0);                                 //close standard input

if(fcntl(fdesc,F_DUPFD,0)== -1)
    perror("fcntl");                       //stdin from FOO now
char buf[ 5 ];
int rc=read(0,buf, 5 );                    //read data from FOO
```

- ✓ The dup and dup2 functions in NI perform the same file duplication function as fcntl. They can be implemented using fcntl as

```

define dup(fd)          fcntl(fd, F_DUPFD, 0)
define dup2(fd, fd2)    close(fd2), fcntl(fd, F_DUPFD, fd2)

```

**Program:**

```

#include<fcntl.h>
#include<stdio.h>
#include<unistd.h>

int main()

{
    int fd, fd2;
    fd = open("s", O_RDONLY | O_CREAT, 0777);
    fd2 = fcntl(fd, F_DUPFD, 0);

    // fd2 = dup(fd); //

    while(1)

```

**❖ lseek:**

- ✓ The lseek function is also used to change the file offset to a different value.
- ✓ Thus lseek allows a process to perform random access of data on any opened file.
- ✓ The prototype of lseek is:

```

#include<sys/types.h>
#include <unistd.h>
off_t lseek(int fd, off_t pos, int whence)

```

- ✓ On success it returns new file offset, and -1 on error.
- ✓ The first argument fd, is an integer file descriptor that refer to an opened file.
- ✓ The second argument pos, specifies a byte offset to be added to a reference location in deriving the new file offset value.
- ✓ The third argument whence, is the reference location.

| Whence value | Reference location            |
|--------------|-------------------------------|
| SEEK_CUR     | Current file pointer address. |
| SEEK_SET     | The beginning of a file.      |
| SEEK_END     | The end of a file.            |

- ✓ They are defined in the header.
- ✓ If an lseek call will result in a new file offset that is beyond the current end of file, two outcomes possible are:
  - If a file is opened for read only, lseek will fail.
  - If a file is opened for write access, lseek will succeed.
  - The data between the end of file and the new file offset address will be initialised with N LL characters.

### Program:

```

include<stdio.h
include<unistd.h
include<sys/stat.h
include<fcntl.h
include<string.h
include<sys/types.h
include<stdlib.h

int main()

    int fd;
    int ret;
    int a[ 0]= , , , ,5, , , , , 0 ;
    fd=open( outfile ,O_CREAT O_APPEND O_RDWR, 0 00);
    int b;
    if(fd<0)

        perror( Open );
        exit( );

    / Writting /
    ret=write(fd, (void )a, sizeof(a));
    if(fd<0)

        perror( Write );
        close(fd);
        exit( );

    / seek sing lseek /
    ret=lseek(fd, sizeof(int), SEEK_SET);
  
```

```

if(fd<0)

    perror( lseek );
    close(fd);
    exit( );

/ read from the position /
ret=read(fd, b,sizeof(int));
if(fd<0)

    perror( read );
    close(fd);
    exit( );

printf( B=%d , b);

close(fd);
return 0;

```

#### ❖ link:

- ✓ The link function creates a new link for the existing file.
- ✓ The prototype of the link function is

```

include<unistd.h

int link(const char  cur_link, const char  new_link);

```

- ✓ If successful, the link function returns 0.
- ✓ If unsuccessful, link returns -1.
- ✓ The first argument cur\_link, is the pathname of existing file.
- ✓ The second argument new\_link is a new pathname to be assigned to the same file.
- ✓ If this call succeeds, the hard link count will be increased by 1.
- ✓ The NI ln command is implemented using the link API

Program:

```

include <stdio.h
include <unistd.h
include <iostream
sing namespace std;
int main(int argc, char  argv)

if(argc = )

cerr<<"usage:"<<argv[0]<<" n";

```

```

return 0;

if(link(argv[ ],argv[ ])== )

    perror("link"); return  ;

return 0;

```

### ❖ **unlink:**

- ✓ The unlink function deletes a link of an existing file.
- ✓ This function decreases the hard link count attributes of the named file, and removes the file name entry of the link from directory file.
- ✓ A file is removed from the file system when its hard link count is zero and no process has any file descriptor referencing that file.
- ✓ The prototype of unlink is

```

include<unistd.h>

int unlink(const char  cur_link);

```

- ✓ If successful, the unlink function returns 0.
- ✓ If unsuccessful, unlink returns - 1.
- ✓ The argument cur\_link is a path name that references an existing file.
- ✓ ANSI C defines the rename function which does the similar unlink operation.
- ✓ The prototype of the rename function is:

```

include<unistd.h>

int rename(const char  old_path_name, const char  new_path_name);

```

- ✓ The NI mv command can be implemented using the link and unlink APIs as shown:

Example:

```

include<unistd.h>
include<string.h>
include<iostream>
using namespace std;
int main ( int argc, char  argv[ ])

if (argc == 2 && strcmp(argv[ ],argv[ ]))
    cerr<<"usage:"<<argv[0]<<" " n";
else if(link(argv[ ],argv[ ]) == 0)
    return unlink(argv[ ]);
return  ;

```

❖ **stat, fstat:**

- ✓ The stat and fstat function retrieves the file attributes of a given file.
- ✓ The only difference between stat and fstat is that the first argument of a stat is a file pathname, where as the first argument of fstat is file descriptor.
- ✓ The prototypes of these functions are:

```
include<unistd.h>

include<sys/stat.h>

int stat(const char  pathname, struct stat  statv);

int fstat(const int fdesc, struct stat  statv);
```

- ✓ The second argument to stat and fstat is the address of a struct stat typed variable which is defined in the header.
- ✓ Its declaration is as follows:

struct stat

```
dev_t      st_dev;      / file system ID /
ino_t      st_ino;      / file inode number /
mode_t     st_mode     / contains file type  Permissions /
nlink_t    st_nlink     / hard link count /
uid_t      st_uid       / file user id /
gid_t      st_gid       / file group id /
dev_t      st_rdev      / contains major  minor number /
off_t      st_size      / file size in bytes /
time_t     st_atime     / last access time /
time_t     st_mtime     / last modification time /
time_t     st_ctime     / last status change time /
;
```

- ✓ The return value of both functions is
  - 0 if they succeed
  - -1 if they fail
  - errno contains an error status code
- ✓ The lstat function prototype is the same as that of stat:
 

```
int lstat(const char  path_name, struct stat  statv);
```
- ✓ We can determine the file type with the macros as shown.

| Macro      | Type of file           |
|------------|------------------------|
| S_ISREG()  | Regular file           |
| S_ISDIR()  | directory file         |
| S_ISCHR()  | character special file |
| S_ISBLK()  | block special file     |
| S_ISFIFO() | pipe or FIFO           |
| S_ISLNK()  | symbolic link          |
| S_ISSOCK() | socket                 |

❖ **access:**

- ✓ The access system call checks the existence and access permission of user to a named file.
- ✓ The prototype of access function is:

```
include<unistd.h>

int access(const char path_name, int flag);
```

- ✓ On success access returns 0, on failure it returns -1.
- ✓ The first argument is the pathname of a file.
- ✓ The second argument flag, contains one or more of the following bit flag.

| Bit Flag | Uses                              |
|----------|-----------------------------------|
| F_OK     | Checks whether a named file exist |
| R_OK     | Test for read permission          |
| W_OK     | Test for write permission         |
| _OK      | Test for execute permission       |

- ✓ The flag argument value to an access call is composed by bitwise ORing one or more of the above bit flags as shown:

```
int rc=access("/usr/SRP/usp.txt",R_OK | W_OK);
```

- ✓ example to check whether a file exists:
 

```
if(access("/usr/SRP/usp.txt", F_OK)== 0)
    printf("file does not exists");
else
    printf("file exists");
```

Program:

```
include<stdio.h>
include<unistd.h>
include<sys/stat.h>
include<sys/types.h>
int main()

{
    char fn[100];
    if(access(fn, F_OK)== 0)
        printf("file does not exists ");
    else
        printf("file exists ");
    int r=access(fn, R_OK | W_OK | _OK );

    printf("Return value is=%d", r);

    return 0;
}
```

❖ **chmod, fchmod:**

- ✓ The chmod and fchmod functions change file access permissions for owner, group others as well as the set\_ ID, set\_GID and sticky flags.
- ✓ A process must have the effective ID of either the super user/owner of the file.
- ✓ The prototypes of these functions are

```
include <sys/stat.h>
include <sys/types.h>
include <unistd.h>
int chmod(const char pathname, mode_t flag);
int fchmod(int fd, mode_t flag);
```

- ✓ The pathname argument of chmod is the path name of a file whereas the fd argument of fchmod is the file descriptor of a file.
- ✓ The chmod function operates on the specified file, whereas the fchmod function operates on a file that has already been opened.
- ✓ To change the permission bits of a file, the effective user ID of the process must be equal to the owner ID of the file, or the process must have super user permissions. The mode is specified as the bitwise OR of the constants shown below.

| Mode    | Description                               |
|---------|-------------------------------------------|
| S_IS ID | set user ID on execution                  |
| S_ISGID | set group ID on execution                 |
| S_IS T  | saved text (sticky bit)                   |
| S_IRW   | read, write, and execute by user (owner)  |
| S_IR SR | read by user (owner)                      |
| S_IW SR | write by user (owner)                     |
| S_I SR  | execute by user (owner)                   |
| S_IRW G | read, write, and execute by group         |
| S_IRGRP | read by group                             |
| S_IWGRP | write by group                            |
| S_I GRP | execute by group                          |
| S_IRW O | read, write, and execute by other (world) |
| S_IROT  | read by other (world)                     |
| S_IWOT  | write by other (world)                    |
| S_I OT  | execute by other (world)                  |

Program:

```
include<sys/types.h
include<sys/stat.h
include<unistd.h
include<stdio.h
```

```
int main()
```

```
char fn[] = "s ";
```

```
int c = chmod(fn, S_IRWXU | S_IRWXG | S_IRWXO);
```

```
printf("File permissions: %d", c);
```

#### ❖ **Chown, fchown, lchown:**

- ✓ The chown functions changes the user ID and group ID of files.
- ✓ The prototypes of these functions are:

```
include <sys/types.h
include <unistd.h
```

```
int chown(const char *path_name, uid_t uid, gid_t gid);
```

```
int fchown(int fd, uid_t uid, gid_t gid);
```

```
int lchown(const char *path_name, uid_t uid, gid_t gid);
```

- ✓ The path\_name argument is the path name of a file.
- ✓ The uid argument specifies the new user ID to be assigned to the file.
- ✓ The gid argument specifies the new group ID to be assigned to the file.

Program:

```
include<stdio.h
include<sys/types.h
include<sys/stat.h
include<unistd.h
include<pwd.h
```

```
int main(int argc, char *argv[])
```

```
{
    if(argc > 1)
```

```

        perror( Arguments count ere wrong );
        return  ;

    struct passwd  pwd = getpwuid(argv[  ]);
    uid_t  ID = pwd  pwd  pw_uid :  ;
    struct stat statv;
    if ( ID == (uid_t)  )
        perror( Inavlid  sername );
    else
        for (int i =  ; i < argc ; i  )
            if (stat(argv[i],  statv)==0)

                if (chown(argv[i],  ID,statv.st_gid))
                    perror (  chown  );
                else
                    perror (  stat  );

    return 0;

```

- ✓ The above program takes at least two command line arguments:
  - The first one is the user name to be assigned to files
  - The second and any subse uent arguments are file path names.
- ✓ The program first converts a given user name to a user ID via getpwuid function. If that succeeds, the program processes each named file as follows: it calls stat to get the file group ID, then it calls chown to change the file user ID. If either the stat or chown fails, error is displayed.

#### ❖ **utime Function:**

- ✓ The utime function modifies the access time and the modification time stamps of a file.
- ✓ The prototype of utime function is

```

include <sys/types.h>
include <unistd.h>
include <utime.h>

```

```

int utime(const char  path_name, struct utimbuf  times);

```

- ✓ On success it returns 0, on failure it returns - .
- ✓ The path\_name argument specifies the path name of a file.
- ✓ The times argument specifies the new access time and modification time for the file.

- ✓ The struct utimbuf is defined in the header as:  
struct utimbuf

```

        time_t actime;           / access time /
        time_t modtime;         / modification time /

```

- ✓ The time\_t datatype is an unsigned long and its data is the number of the seconds elapsed since the birthday of NI : A , an of 0.
- ✓ If the times (variable) is specified as N LL, the function will set the named file access and modification time to the current time.
- ✓ If the times (variable) is an address of the variable of the type struct utimbuf, the function will set the file access time and modification time to the value specified by the variable.

#### ❖ File & Record Locking:

- ✓ Multiple processes perform read and write operation on the same file concurrently.
- ✓ This provides a means for data sharing among processes, but it also renders difficulty for any process in determining when the other process can override data in a file.
- ✓ So, in order to overcome this drawback NI and POSI standard support file locking mechanism.
- ✓ File locking is applicable for regular files.
- ✓ Only a process can impose a write lock or read lock on either a portion of a file or on the entire file.
- ✓ The differences between the read lock and the write lock is that when write lock is set, it prevents the other process from setting any overlapping read or write lock on the locked file.
- ✓ Similarly when a read lock is set, it prevents other processes from setting any overlapping write locks on the locked region.
- ✓ The intention of the write lock is to prevent other processes from both reading and writing the locked region while the process that sets the lock is modifying the region, so write lock is termed as "Exclusive lock".
- ✓ The use of read lock is to prevent other processes from writing to the locked region while the process that sets the lock is reading data from the region.
- ✓ Other processes are allowed to lock and read data from the locked regions. Hence a read lock is also called as "shared lock".
- ✓ File lock may be mandatory if they are enforced by an operating system kernel.

- ✓ If a mandatory exclusive lock is set on a file, no process can use the read or write system calls to access the data on the locked region.
- ✓ These mechanisms can be used to synchronize reading and writing of shared files by multiple processes.
- ✓ If a process locks up a file, other processes that attempt to write to the locked regions are blocked until the former process releases its lock.
- ✓ Problem with mandatory lock is – if a runaway process sets a mandatory exclusive lock on a file and never unlocks it, then, no other process can access the locked region of the file until the runaway process is killed or the system has to be rebooted.
- ✓ If locks are not mandatory, then it has to be advisory lock.
- ✓ A kernel at the system call level does not enforce advisory locks.
- ✓ This means that even though a lock may be set on a file, no other processes can still use the read and write functions to access the file.
- ✓ To make use of advisory locks, process that manipulate the same file must co operate such that they follow the given below procedure for every read or write operation to the file.
  - Try to set a lock at the region to be accesses. If this fails, a process can either wait for the lock re uest to become successful.
  - After a lock is ac uired successfully, read or write the locked region.
  - Release the lock.
- ✓ If a process sets a read lock on a file, for example from address 0 to 5 , then sets a write lock on the file from address 0 to 5 , the process will own only one write lock on the file from 0 to 5 , the previous read lock from 0 to 5 is now covered by the write lock and the process does not own two locks on the region from 0 to 5 . This process is called “Lock Promotion”.
- ✓ Furthermore, if a process now unblocks the file from to 0, it will own two write locks on the file: one from 0 to and the other from to 5 . This process is called “Lock Splitting”.
- ✓ NI systems provide fcntl function to support file locking. By using fcntl it is possible to impose read or write locks on either a region or an entire file.
- ✓ The prototype of fcntl is

```
include <fcntl.h>

int fcntl(int fdesc, int cmd_flag,.....);
```

- ✓ The first argument specifies the file descriptor.
- ✓ The second argument cmd\_flag specifies what operation has to be performed.
- ✓ If fcntl is used for file locking then it can values as

|          |                                                                     |
|----------|---------------------------------------------------------------------|
| F_SETLK  | sets a file lock, do not block if this cannot succeed immediately.  |
| F_SETLKW | sets a file lock and blocks the process until the lock is acquired. |
| F_GETLK  | queries as to which process locked a specified region of file.      |

- ✓ For file locking purpose, the third argument to fcntl is an address of a struct flock type variable.
  - ✓ This variable specifies a region of a file where lock is to be set, unset or queried.
- struct flock

```

short l_type;      / what lock to be set or to unlock file /
short l_whence;    / Reference address for the next field /
off_t l_start ;    / offset from the l_whence reference addr /
off_t l_len ;      / how many bytes in the locked region /
pid_t l_pid ;      / pid of a process which has locked the file /
;

```

- ✓ The l\_type field specifies the lock type to be set or unset.
- ✓ The possible values, which are defined in the header, and their uses are

| <b>l_type value</b> | <b>Uses</b>                            |
|---------------------|----------------------------------------|
| F_RDLCK             | Set a read lock on a specified region  |
| F_WRLCK             | Set a write lock on a specified region |
| F_UNLCK             | Unlocked region F_UNLCK                |

- ✓ The l\_whence, l\_start, l\_len define a region of a file to be locked or unlocked.
- ✓ The possible values of l\_whence and their uses are

| <b>l_whence value</b> | <b>Uses</b>                                                |
|-----------------------|------------------------------------------------------------|
| SEEK_CUR              | The l_start value is added to current file pointer address |
| SEEK_SET              | The l_start value is added to byte 0 of the file           |
| SEEK_END              | The l_start value is added to the end of the file          |

- ✓ A lock set by the fcntl API is an advisory lock but we can also use fcntl for mandatory locking purpose with the following attributes set before using fcntl
  - Turn on the set GID flag of the file.
  - Turn off the group execute right permission of the file.
- ✓ In the given example program we have performed a read lock on a file "Raj" from the 0th byte to 5th byte.

Program:

```

include<stdio.h
include<fcntl.h
include<string.h
include<unistd.h
include<iostream
using namespace std;

int main(int argc, char argv[])

    char file=argv[ ];
    int fd;

    struct flock lock;
    cout<< I am openening the file <<file<<endl;
    fd=open(file, O_WRONLY);
    cout<< I am locking it Buddy <<endl;
    memset( lock, 0, sizeof(lock));
    lock.l_type=F_WRLCK;
    fcntl(fd, F_SETLKW, lock);
    cout<< I locked the file, Press Enter to unlock <<endl;
    getchar();
    cout<< Releasing Buddy ;
    lock.l_type=F_ NLCK;
    fcntl(fd,F_SETLKW, lock);
    close(fd);
    return 0;

```

**❖ Directory File APIs:**

- ✓ A Directory file is a record oriented file, where each record stores a file name and the inode number of a file that resides in that directory.
- ✓ Directories are created with the mkdir API and deleted with the rmdir API.
- ✓ The prototype of mkdir is

```
include <sys/stat.h>
include <unistd.h>

int mkdir(const char path_name, mode_t mode);
```

- ✓ The first argument is the path name of a directory file to be created.
- ✓ The second argument mode, specifies the access permission for the owner, groups and others to be assigned to the file. This function creates a new empty directory.
- ✓ The entries for "." and ".." are automatically created. The specified file access permission, mode, are modified by the file mode creation mask of the process.
- ✓ To allow a process to scan directories in a file system independent manner, a directory record is defined as struct dirent in the header for NI .
- ✓ Some of the functions that are defined for directory file operations in the above header are

```
include <sys/types.h>
if defined (BSD) _POSIX_SOURCE
    include <sys/dir.h>
    typedef struct dirent Dirent;
else
    include <dirent.h>
    typedef struct dirent Dirent;
endif

DIR opendir(const char path_name);
Dirent readdir(DIR dir_fd);
int closedir(DIR dir_fd);
void rewinddir(DIR dir_fd);
```

- ✓ The uses of these functions are:

| Function  | se                                                                                                                                                       |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| opendir   | Opens a directory file for read only. Returns a file handle dir for future reference of the file.                                                        |
| readdir   | Reads a record from a directory file referenced by dir fdesc and returns that record information.                                                        |
| rewinddir | Resets the file pointer to the beginning of the directory file referenced by dirfdesc. The next call to readdir will read the first record from the file |
| closedir  | closes a directory file referenced by dir fdesc.                                                                                                         |

- ✓ An empty directory is deleted with the rmdir API.
- ✓ The prototype of rmdir is

```
include <unistd.h>

int rmdir(const char path_name);
```

- ✓ If the link count of the directory becomes 0, with the call and no other process has the directory open then the space occupied by the directory is freed.
- ✓ NI systems have defined additional functions for random access of directory file records.

| Function | se                                                                  |
|----------|---------------------------------------------------------------------|
| telldir  | Returns the file pointer of a given dir_fdsc                        |
| seekdir  | Changes the file pointer of a given dir_fdsc to a specified address |

Program:

```
include<stdio.h>
include<sys/types.h>
include<unistd.h>
include<string.h>
include<sys/stat.h>
include<iostream>
using namespace std;
if defined(BSD) _POSIX _SO _RCE
include<sys/dir.h>
typedef struct dirent Dirent;
else
include<dirent.h>
typedef struct dirent Dirent;
endif
```

```

int main(int argc, char argv[])

Dirent dp;
int i=0, cnt=0;
DIR dir_fdesc;
while( argc 0)

if( (dir_fdesc=opendir( argv)))

if(mkdir( argv,S_IRWXU | S_IRWXG | S_IRWXO)== 0)
perror( opendir );
continue;

for(i=0;i< argc;i++)
for(cnt=0;dp=readdir(dir_fdesc);)

if(i)
cout<<dp->d_name<<endl;
if(strcmp(dp->d_name, ".") != strcmp(dp->d_name, ".."))
cnt++;

if( cnt)

rmdir( argv);
break;

rewinddir(dir_fdesc);

closedir(dir_fdesc);

```

#### ❖ Device File API:

- ✓ Device files are used to interface physical device with application programs.
- ✓ A process with superuser privileges to create a device file must call the mknod API.
- ✓ The user ID and group ID attributes of a device file are assigned in the same manner as for regular files.
- ✓ When a process reads or writes to a device file, the kernel uses the major and minor device numbers of a file to select a device driver function to carry out the actual data transfer.
- ✓ Device file support is implementation dependent. NI System defines the mknod API to create device files.
- ✓ The prototype of mknod is

```

include <unistd.h>
include <sys/stat.h>

int mknod(const char path_name, mode_t mode, int device_id);

```

- ✓ The first argument pathname is the pathname of a device file to be created.
- ✓ The second argument mode specifies the access permission, for the owner, group and others, also S\_IFC R or S\_IBLK flag to be assigned to the file.
- ✓ The third argument device\_id contains the major and minor device number.
- ✓ Example
 

```

mknod("SCSI5",S_IFBLK S_IRW S_IRW G
S_IRW O,( 5<< ) );

```
- ✓ The above function creates a block device file "divya", to which all the three i.e. read, write and execute permission is granted for user, group and others with major number as and minor number .
- ✓ On success mknod API returns 0 , else it returns .

Program:

```

include<stdio.h>
include<stdlib.h>
include<sys/types.h>
include<unistd.h>
include<fcntl.h>
include<sys/stat.h>
include<iostream>
using namespace std;
int main(int argc, char argv[])

    if(argc = )

        cout<< usage: <<argv[0]<< <file <major_no <minor_no ;
        return 0;

    int major=atoi(argv[ ]),minor=atoi(argv[ ]);
    (void) mknod(argv[ ], S_IFC R S_IRW S_IRW G
S_IRW O, (major<< ) minor);
    int rc= ,fd=open(argv[ ],O_RDWR O_NONBLOCK
O_NOCTTY);
    char buf[ 5 ];
    while(rc fd = )
        if((rc=read(fd,buf,sizeof(buf)))<0)

```

```

        perror( read );
    else if(rc)
        cout<<buf<<endl;
    else
        cout<<buf<<endl;
    close(fd);

```

#### ❖ FIFO File API:

- ✓ FIFO files are sometimes called named pipes.
- ✓ Pipes can be used only between related processes when a common ancestor has created the pipe.
- ✓ Creating a FIFO is similar to creating a file.
- ✓ Indeed the pathname for a FIFO exists in the file system.
- ✓ The prototype of mkfifo is

```

include <unistd.h>
include <sys/stat.h>
include <sys/types.h>

```

```

int mkfifo(const char path_name, mode_t mode);

```

- ✓ The first argument pathname is the pathname(filename) of a FIFO file to be created.
- ✓ The second argument mode specifies the access permission for user, group and others and as well as the S\_IFIFO flag to indicate that it is a FIFO file.
- ✓ On success it returns 0 and on failure it returns -1.
- ✓ Example
 

```
mkfifo("FIFO5",S_IFIFO | S_IRW | S_IRGRP | S_IXGRP | S_IXOT);
```
- ✓ The above statement creates a FIFO file "divya" with read write execute permission for user and only read permission for group and others.
- ✓ Once we have created a FIFO using mkfifo, we open it using open.
- ✓ Indeed, the normal file I/O functions (read, write, unlink etc) all work with FIFOs.
- ✓ When a process opens a FIFO file for reading, the kernel will block the process until there is another process that opens the same file for writing.
- ✓ Similarly, whenever a process opens a FIFO file write, the kernel will block the process until another process opens the same FIFO for reading.

- ✓ This provides a means for synchronization in order to undergo inter process communication.
- ✓ If a particular process tries to write something to a FIFO file that is full, then that process will be blocked until another process has read data from the FIFO to make space for the process to write.
- ✓ Similarly, if a process attempts to read data from an empty FIFO, the process will be blocked until another process writes data to the FIFO.
- ✓ From any of the above condition if the process doesn't want to get blocked then we should specify O\_NONBLOCK in the open call to the FIFO file.
- ✓ If the data is not ready for read/write then open returns - instead of process getting blocked.
- ✓ If a process writes to a FIFO file that has no other process attached to it for read, the kernel will send SIGPIPE signal to the process to notify that it is an illegal operation.
- ✓ Another method to create FIFO files (not exactly) for inter process communication is to use the pipe system call.
- ✓ The prototype of pipe is:

```
include <unistd.h>

int pipe(int fds[ 2]);
```

- ✓ Returns 0 on success and - on failure.
- ✓ If the pipe call executes successfully, the process can read from fd[0] and write to fd[1]. A single process with a pipe is not very useful. usually a parent process uses pipes to communicate with its children.

Program:

```
include<stdio.h>
include<sys/types.h>
include<unistd.h>
include<fcntl.h>
include<sys/stat.h>
include<string.h>
include<errno.h>
include<iostream>
using namespace std;

int main(int argc,char argv[])

    if(argc = 2)
        argc = 2;
```

```

        cout<< usage: <<argv[0]<< <file [ <arg ] ;
        return 0;

        int fd;
        char buf[ 5 ];
        (void) mkfifo(argv[ ], S_IFIFO S_IRW S_IRW G
S_IRW O );
        if(argc== )

                fd=open(argv[ ],O_RDONLY O_NONBLOCK);
                while(read(fd,buf,sizeof(buf))!=
errno==EAGAIN)
                sleep( );
                while(read(fd,buf,sizeof(buf)) 0)
                cout<<buf<<endl;

        else

                fd=open(argv[ ],O_WRONLY);
                write(fd,argv[ ],strlen(argv[ ]));

        close(fd);

```

#### ❖ **Symbolic Link File APIs:**

- ✓ A symbolic link is an indirect pointer to a file, unlike the hard links which pointed directly to the inode of the file.
- ✓ Symbolic links are developed to get around the limitations of hard links:
  - Symbolic links can link files across file systems.
  - Symbolic links can link directory files.
  - Symbolic links always reference the latest version of the files to which they link.
  - There are no file system limitations on a symbolic link and what it points to and anyone can create a symbolic link to a directory.
  - Symbolic links are typically used to move a file or an entire directory hierarchy to some other location on a system.
  - A symbolic link is created with the symlink.

- ✓ The prototype is:

```
include <unistd.h>
include <sys/types.h>
include <sys/stat.h>

int symlink(const char org_link, const char sym_link);
int readlink(const char sym_link, char buf, int size);
int lstat(const char sym_link, struct stat statv);
```

- ✓ The org\_link and sym\_link arguments to a sym\_link call specify the original file path name and the symbolic link path name to be created.

Program:

```
include<unistd.h>
include<sys/types.h>
include<string.h>
include<stdio.h>
int main(int argc, char argv[])

    char buf [ 5 ], tname [ 5 ];
    if (argc == )
        return symlink(argv[ ], argv[ ]); / create a symbolic
link /
    else
        return link(argv[ ], argv[ ]); / creates a hard link /
```

## OD LE 0

### NI PROCESSES

#### Introduction:

A Process is a program under execution in a NI or POSI system.

#### main Function:

A C program starts execution with a function called main. The prototype for the main function is:

```
int main(int argc, char argv[]);
```

where argc is the number of command line arguments, and argv is an array of pointers to the arguments.

When a C program is executed by the kernel by one of the exec functions, a special start up routine is called before the main function is called. The executable program file specifies this routine as the starting address for the program; this is set up by the link editor when it is invoked by the C compiler. This start up routine takes values from the kernel, the command line arguments and the environment and sets things up so that the main function is called.

#### Process Termination:

There are eight ways for a process to terminate. Normal termination occurs in five ways:

- ✓ Return from main.
- ✓ Calling exit.
- ✓ Calling \_exit or \_Exit.
- ✓ Return of the last thread from its start routine.
- ✓ Calling pthread\_exit from the last thread.

Abnormal termination occurs in three ways:

- ✓ Calling abort.
- ✓ Receipt of a signal.
- ✓ Response of the last thread to a cancellation request.

#### Exit Functions:

Three functions terminate a program normally: \_exit and \_Exit, which return to the kernel immediately, and exit, which performs certain cleanup processing and then returns to the kernel.

```
include <stdlib.h>
void exit(int status);
void _Exit(int status);

include <stdlib.h>
void _exit(int status);
```

All three exit functions expect a single integer argument, called the exit status. Returning an integer value from the main function is equivalent to calling exit with the same value. Thus exit(0) is the same as return(0) from the main function.

In the following situations the exit status of the process is undefined.

- ✓ any of these functions is called without an exit status.
- ✓ main does a return without a return value.
- ✓ main “falls off the end”, i.e if the exit status of the process is undefined.

Example:

```
cc Pgm.c
./a.out hello, world
echo // Print the exit status
```

### atexit Function:

With ISO C, a process can register up to functions that are automatically called by exit. These are called exit handlers and are registered by calling the atexit function.

```
include <stdlib.h>

int atexit(void ( func) (void))
```

Returns: 0 if OK, nonzero on error

This declaration says that we pass the address of a function as the argument to atexit. When this function is called, it is not passed any arguments and is not expected to return a value. The exit function calls these functions in reverse order of their registration. Each function is called as many times as it was registered.

Program:

```
include<stdlib.h
include <apue. e/include/apue.h
include<iostream
using namespace std;
static void my_exit (void);
static void my_exit (void);
static void my_exit (void);
static void my_exit (void);
int main(void)

if (atexit(my_exit ) = 0)
perror( can t register my_exit );
if (atexit(my_exit ) = 0)
perror( can t register my_exit );
if (atexit(my_exit ) = 0)
perror( can t register my_exit );
if (atexit(my_exit ) = 0)
perror( can t register my_exit );
printf( main is done n );
return(0);

static void my_exit (void)

cout<< Fourth exit handler <<endl;

static void my_exit (void)

cout<< third exit handler <<endl;

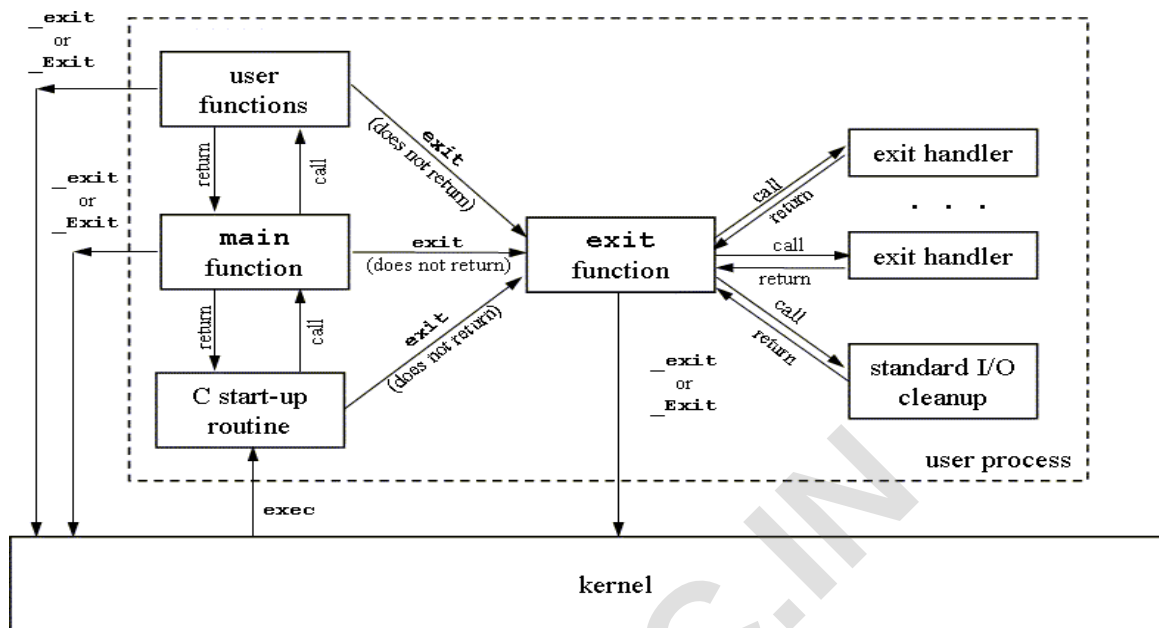
static void my_exit (void)

cout<< first exit handler <<endl;

static void my_exit (void)

cout<< second exit handler <<endl;
```

The following figure summarizes how a C program is started and the various ways it can terminate.



### Command Line arguments:

When a program is executed, the process that does the exec can pass command line arguments to the new program.

Example: Echo all command line arguments to standard output

```
int main(int argc, char argv[])

    int i;
    for(i=0; i<argc; i++)
        printf("Argv[%d]: %s\n", i, argv[i]);
    exit(0);
```

```
gcc Pgm .c
./a.out i ello Dear ow are you
argv[0]=./a.out
Argv[ ]= i
Argv[ ]= ello
Argv[ ]=Dear
.....
```

### Environment List:

Each program is also passed an environment list. Like the argument list, the environment list is an array of character pointers, with each pointer containing the address of a null terminated C string.

The address of the array of pointers is contained in the global variable `environ`:

```
extern char **environ;
```

Generally any environmental variable is of the form: `name=value`.



Fig: Environment consisting of five C character strings

### Memory layout of C Program:

historically, a C program has been composed of the following pieces:

- **Text segment:** the machine instructions that the CP executes. Usually, the text segment is sharable so that only a single copy needs to be in memory for frequently executed programs, such as text editors, the C compiler, the shells, and so on. Also, the text segment is often read only, to prevent a program from accidentally modifying its instructions.
- **Initialized data segment:** usually called simply the data segment, containing variables that are specifically initialized in the program. For example, the C declaration

```
int maxcount = 0;
```

appearing outside any function causes this variable to be stored in the initialized data segment with its initial value.

- **Uninitialized data segment:** often called the `bss` segment, named after an ancient assembler operator that stood for `block started by symbol`. Data in this segment is initialized by the kernel to arithmetic 0 or null pointers before the program starts executing. The C declaration
 

```
long sum[ 000];
```

 appearing outside any function causes this variable to be stored in the uninitialized data segment.
- **Stack:** where automatic variables are stored, along with information that is saved each time a function is called. Each time a function is called, the address of where to return to and certain information about the caller's environment, such as some of the machine registers, are saved on the stack. The newly called function then allocates room on the stack for its automatic and temporary variables. This is how recursive functions in C can work. Each time a recursive function calls itself, a new stack frame is used, so one set of variables doesn't interfere with the variables from another instance of the function.
- **Heap:** where dynamic memory allocation usually takes place. Historically, the heap has been located between the uninitialized data and the stack.

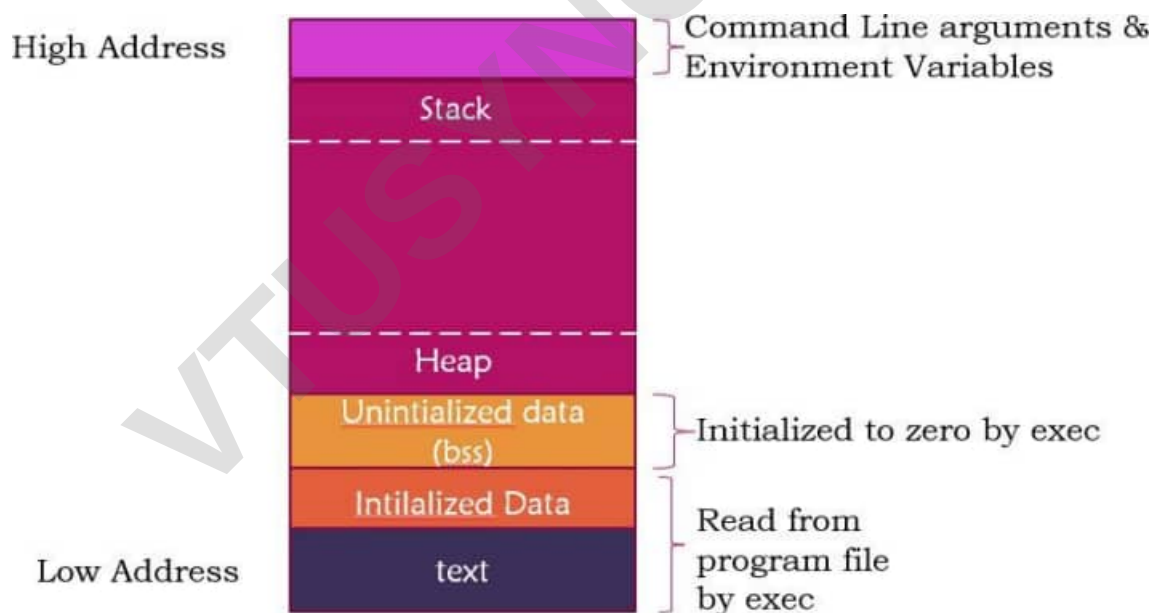


Fig: Memory Layout of C Program

### Shared Libraries:

Nowadays most NI systems support shared libraries. Shared libraries remove the common library routines from the executable file, instead maintaining a single copy of the library routine somewhere in memory that all processes reference. This reduces the size of each executable file but may add some runtime overhead, either when the program is first executed or the first time each shared library function is called. Another advantage of shared libraries is that, library functions can be replaced with new versions without having to re-link, edit every program that uses

the library. With cc compiler we can use the option -g to indicate that we are using shared library.

### Memory Allocation:

ISO C specifies three functions for memory allocation:

- **malloc**, which allocates a specified number of bytes of memory. The initial value of the memory is indeterminate.
- **calloc**, which allocates space for a specified number of objects of a specified size. The space is initialized to all 0 bits.
- **realloc**, which increases or decreases the size of a previously allocated area. When the size increases, it may involve moving the previously allocated area somewhere else, to provide the additional room at the end. Also, when the size increases, the initial value of the space between the old contents and the end of the new area is indeterminate.

```
#include <stdlib.h>

void *malloc(size_t size);
void *calloc(size_t nobj, size_t size);
void *realloc(void *ptr, size_t newsiz);
```

All three return: non null pointer if OK, NULL on error.

```
void free(void *ptr);
```

The pointer returned by the three allocation functions is guaranteed to be suitably aligned so that it can be used for any data object. Because the three alloc functions return a generic void pointer, if we include (to obtain the function prototypes), we do not explicitly have to cast the pointer returned by these functions when we assign it to a pointer of a different type.

The function free causes the space pointed to by ptr to be deallocated. This freed space is usually put into a pool of available memory and can be allocated in a later call to one of the three alloc functions.

The realloc function lets us increase or decrease the size of a previously allocated area. For example, if we allocate room for 5 elements in an array that we fill in at runtime but find that we need room for more than 5 elements, we can call realloc. If there is room beyond the end of the existing region for the requested space, then realloc doesn't have to move anything; it simply allocates the additional area at the end and returns the same pointer that we passed it. But if there isn't room at the end of the existing region,

realloc allocates another area that is large enough, copies the existing 5 element array to the new area, frees the old area, and returns the pointer to the new area.

The allocation routines are usually implemented with the sbrk( ) system call. Although sbrk can expand or contract the memory of a process, most versions of malloc and free never decrease their memory size. The space that we free is available for a later allocation, but the freed space is not usually returned to the kernel; that space is kept in the malloc pool.

It is important to realize that most implementations allocate a little more space than is requested and use the additional space for record keeping the size of the allocated block, a pointer to the next allocated block, and the like. This means that writing past the end of an allocated area could overwrite this record keeping information in a later block. These types of errors are often catastrophic, but difficult to find, because the error may not show up until much later. Also, it is possible to overwrite this record keeping by writing before the start of the allocated area.

Because memory allocation errors are difficult to track down, some systems provide versions of these functions that do additional error checking every time one of the three alloc functions or free is called. These versions of the functions are often specified by including a special library for the link editor. There are also publicly available sources that you can compile with special flags to enable additional runtime checking.

### **Alternate Memory Allocators:**

any replacements for malloc and free are available.

#### ➤ **libmalloc:**

S R based systems, such as Solaris, include the libmalloc library, which provides a set of interfaces matching the ISO C memory allocation functions. The libmalloc library includes mallopt, a function that allows a process to set certain variables that control the operation of the storage allocator. A function called mallinfo is also available to provide statistics on the memory allocator.

#### ➤ **vmalloc:**

o describes a memory allocator that allows processes to allocate memory using different techniques for different regions of memory. In addition to the functions specific to vmalloc, the library also provides emulations of the ISO C memory allocation functions.

➤ **quick-fit:**

historically, the standard malloc algorithm used either a best fit or a first fit memory allocation strategy. quick fit is faster than either, but tends to use more memory. Free implementations of malloc and free based on quick fit are readily available from several FTP sites.

➤ **alloca Function:**

The function alloca has the same calling sequence as malloc; however, instead of allocating memory from the heap, the memory is allocated from the stack frame of the current function. The advantage is that we don't have to free the space; it goes away automatically when the function returns. The alloca function increases the size of the stack frame. The disadvantage is that some systems can't support alloca, if it's impossible to increase the size of the stack frame after the function has been called.

**Environment variables:**

The environment strings are usually of the form: name=value. The UNIX kernel never looks at these strings; their interpretation is up to the various applications. The shells, for example, use numerous environment variables. Some, such as HOME and USER, are set automatically at login, and others are for us to set. We normally set environment variables in a shell start up file to control the shell's actions. The functions that we can use to set and fetch values from the variables are setenv, putenv, and getenv functions. The prototype of these functions are:

```
#include <stdlib.h>

char *getenv(const char *name);
```

Returns: pointer to value associated with name, NULL if not found.

Note that this function returns a pointer to the value of a name=value string. We should always use getenv to fetch a specific value from the environment, instead of accessing environ directly. In addition to fetching the value of an environment variable, sometimes we may want to set an environment variable. We may want to change the value of an existing variable or add a new variable to the environment. The prototypes of these functions are

```
#include <stdlib.h>

int putenv(char *str);

int setenv(const char *name, const char *value, int rewrite);

int unsetenv(const char *name);
```

All return: 0 if OK, nonzero on error.

- The putenv function takes a string of the form name=value and places it in the environment list. If name already exists, its old definition is first removed.
- The setenv function sets name to value. If name already exists in the environment, then
  - ✓ if rewrite is nonzero, the existing definition for name is first removed;
  - ✓ if rewrite is 0, an existing definition for name is not removed, name is not set to the new value, and no error occurs.
- The unsetenv function removes any definition of name. It is not an error if such a definition does not exist.

Note the difference between putenv and setenv. Whereas setenv must allocate memory to create the name=value string from its arguments, putenv is free to place the string passed to it directly into the environment.

Environment variables defined in the Single NI Specification

| Variable    | Description                                         |
|-------------|-----------------------------------------------------|
| COL NS      | terminal width                                      |
| DATE SK     | getdate( ) template file pathname                   |
| O E         | home directory                                      |
| LANG        | name of locale                                      |
| LC_ALL      | name of locale                                      |
| LC_COLLATE  | name of locale for collation                        |
| LC_CTYPE    | name of locale for character classification         |
| LC_ ESSAGES | name of locale for messages                         |
| LC_ ONETARY | name of locale for monetary editing                 |
| LC_N ERIC   | name of locale for numeric editing                  |
| LC_TI E     | name of locale for date/time formatting             |
| LINES       | terminal height                                     |
| LOGNA E     | login name                                          |
| SG ERB      | fmtmsg( ) message components to process             |
| NLSPAT      | sequence of templates for message catalogs          |
| PAT         | list of path prefixes to search for executable file |
| PWD         | absolute pathname of current working directory      |
| S ELL       | name of user's preferred shell                      |
| TER         | terminal type                                       |
| T PDIR      | pathname of directory for creating temporary files  |
| T           | time zone information                               |

**Note:**

- If we re modifying an existing name:
  - ✓ If the size of the new value is less than or e ual to the size of the existing value, we can just copy the new string over the old string.
  - ✓ If the size of the new value is larger than the old one, however, we must malloc to obtain room for the new string, copy the new string to this area, and then replace the old pointer in the environment list for name with the pointer to this allocated area.
- If we re adding a new name, it s more complicated. First, we have to call malloc to allocate room for the name=value string and copy the string to this area.
  - ✓ Then, if it s the first time we ve added a new name, we have to call malloc to obtain room for a new list of pointers. We copy the old environment list to this new area and store a pointer to the name=value string at the end of this list of pointers. We also store a null pointer at the end of this list, of course. Finally, we set environ to point to this new list of pointers.
  - ✓ If this isn t the first time we ve added new strings to the environment list, then we know that we ve already allocated room for the list on the heap, so we just call realloc to allocate room for one more pointer. The pointer to the new name=value string is stored at the end of the list (on top of the previous null pointer), followed by a null pointer.

**Program:**

```

include<stdio.h
include<stdlib.h

int main(void)

    char path;

    path=getenv( INCL DE );
    if(path =N LL)
        printf( INCL DE=%s , path); / /usr/include /

    if(putenv( INCL DE=//5/usr/inlude ) =0)
        printf( putenv failed ); / //5/usr/include /

    path=getenv( INCL DE );
    if(path =N LL)
        printf( INCL DE=%s , path);

    if(putenv( INCL DE ) =0)
        printf( putenv failed );

```

```
return(E IT_S CCESS);
```

### setjmp & longjmp Functions:

In C, we can't goto a label that's in another function. Instead, we must use the setjmp and longjmp functions to perform this type of branching. As we'll see, these two functions are useful for handling error conditions that occur in a deeply nested function call.

```
#include <setjmp.h>
int setjmp(jmp_buf env);
```

The setjmp function records or marks a location in a program code so that later when the longjmp function is called from some other function, the execution continues from the location onwards. The env variable (the first argument) records the necessary information needed to continue execution. The env is of the jmp\_buf defined in file, it contains the task.

Program:

```
include <stdio.h>
include <stdlib.h>
include <setjmp.h>
include <iostream>
using namespace std;

/ declare variable of type jmp_buf /
jmp_buf resume_here;

void hello(void);

int main(void)

int ret_val;

/ Initialize resume_here by calling setjmp() /
if (setjmp(resume_here))
    printf( After longjump(), back in main() );
    cout<<endl;
    printf( jump buffer variable resume_here becomes INVALID );
    cout<<endl;

else
    printf( setjmp() returns first time );
    cout<<endl;
```

```
hello();
```

```
return 0;
```

```
void hello(void)
```

```
printf( "hey, I m in hello() ");
```

```
longjmp(resume_here, 1);
```

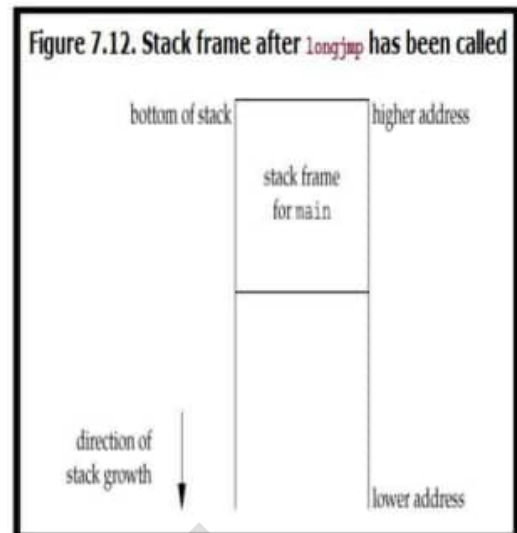
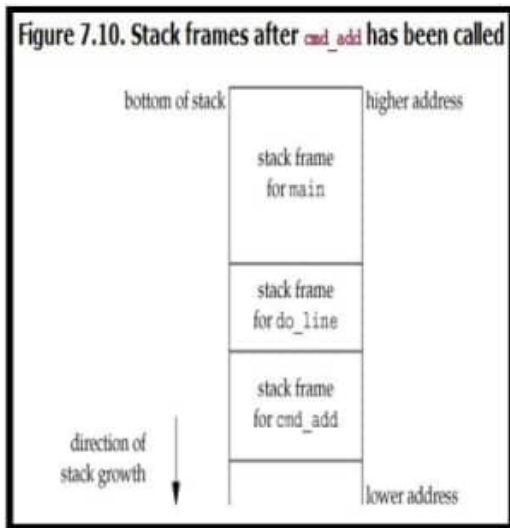
```
/* other code */
```

```
printf( "can't be reached here because I did longjmp\n");
```

- The setjmp function always returns '0' on its success when it is called directly in a process (for the first time).
- The longjmp function is called to transfer a program flow to a location that was stored in the env argument.
- The program code marked by the env must be in a function that is among the callers of the current function.
- When the process is jumping to the target function, all the stack space used in the current function and its callers, upto the target function are discarded by the longjmp function.
- The process resumes execution by re executing the setjmp statement in the target function that is marked by env. The return value of setjmp function is the value(val), as specified in the longjmp function call.
- The 'val' should be nonzero, so that it can be used to indicate where and why the longjmp function was invoked and process can do error handling accordingly.

**Note:**

The values of automatic and register variables are indeterminate when the longjmp is called but static and global variable are unaltered. The variables that we don't want to roll back after longjmp are declared with keyword 'volatile'.



### getrlimit & setrlimit Functions:

Every process has a set of resource limits, some of which can be queried and changed by the `getrlimit` and `setrlimit` functions.

```
#include<sys/resource.h>

int getrlimit(int resource, struct rlimit *rlptr);

int setrlimit(int resource, const struct rlimit *rlptr);
```

Both return: 0 if OK, nonzero on error.

Each call to these two functions specifies a single resource and a pointer to the following structure:

```
struct rlimit
{
    rlim_t rlim_cur;    //soft limit: current limit
    rlim_t rlim_max;    //hard limit: maximum value for rlim_cur
}
```

Three rules govern the changing of the resource limits.

- ✓ A process can change its soft limit to a value less than or equal to its hard limit.
- ✓ A process can lower its hard limit to a value greater than or equal to its soft limit. This lowering of the hard limit is irreversible for normal users.
- ✓ Only a superuser process can raise a hard limit.

An infinite limit is specified by the constant `RLIMIT_INFINITY`.

| Variable                    | Description                                                                                                                                                                                                |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>RLIMIT_AS</code>      | The maximum size in bytes of a process's total available memory.                                                                                                                                           |
| <code>RLIMIT_CORE</code>    | The maximum size in bytes of a core file. A limit of 0 prevents the creation of a core file.                                                                                                               |
| <code>RLIMIT_CPU</code>     | The maximum amount of CPU time in seconds. When the soft limit is exceeded, the <code>SIGCPU</code> signal is sent to the process.                                                                         |
| <code>RLIMIT_DATA</code>    | The maximum size in bytes of the data segment: the sum of the initialized data, uninitialized data, and heap.                                                                                              |
| <code>RLIMIT_FSIZE</code>   | The maximum size in bytes of a file that may be created. When the soft limit is exceeded, the process is sent the <code>SIGFSE</code> signal. <code>RLIMIT_LOCKS</code> The maximum number of file locks a |
| <code>RLIMIT_MEMLOCK</code> | The maximum amount of memory in bytes that a process can lock into memory using <code>mlock()</code> .                                                                                                     |
| <code>RLIMIT_NOFILE</code>  | The maximum number of open files per process. Changing this limit affects the value returned by the <code>sysconf</code> function for its <code>_SC_OPEN_MAX</code> argument.                              |
| <code>RLIMIT_NPROC</code>   | The maximum number of child processes per real user ID. Changing this limit affects the value returned for <code>_SC_CHILD_MAX</code> by the <code>sysconf</code> function.                                |
| <code>RLIMIT_RSS</code>     | Maximum resident set size (RSS) in bytes. If available physical memory is low, the kernel takes memory from processes that exceed their RSS.                                                               |
| <code>RLIMIT_SBSIZE</code>  | The maximum size in bytes of socket buffers that a user can consume at any given time.                                                                                                                     |
| <code>RLIMIT_STACK</code>   | The maximum size in bytes of the stack.                                                                                                                                                                    |
| <code>RLIMIT_</code>        | This is a synonym for <code>RLIMIT_AS</code> .                                                                                                                                                             |

The resource limits affect the calling process and are inherited by any of its children. This means that the setting of resource limits needs to be built into the shells to affect all our future processes.

Program: To print the current resource limits.

```

#include<stdio.h>
#include <unistd.h>/include/apue.h
if defined(BSD) defined( _ACOS)
include <sys/time.h>
define F_T %lld
else
define F_T %ld
endif
include <sys/resource.h>

```

```

define doit(name) pr_limits( name, name)
include<iostream
using namespace std;

```

```

static void pr_limits(char , int);
int main(void)

```

```

    ifdef RLI IT_AS
doit(RLI IT_AS);
    endif
doit(RLI IT_CORE);
doit(RLI IT_CP );
doit(RLI IT_DATA);
doit(RLI IT_FSI E);
    ifdef RLI IT_LOCKS
doit(RLI IT_LOCKS);
    endif
    ifdef RLI IT_ E LOCK
doit(RLI IT_ E LOCK);
    endif
doit(RLI IT_NOFILE);
    ifdef RLI IT_NPROC
doit(RLI IT_NPROC);
    endif
    ifdef RLI IT_RSS
doit(RLI IT_RSS);
    endif
    ifdef RLI IT_SBSI E
doit(RLI IT_SBSI E);
    endif
doit(RLI IT_STACK);
    ifdef RLI IT_ E
doit(RLI IT_ E );
    endif
exit(0);

```

```

static void pr_limits(char name, int resource)

```

```

    struct rlimit limit;
    if (getrlimit(resource, limit) < 0)
err_sys( getrlimit error for %s , name);
    printf( % s , name);
    if (limit.rlim_cur == RLI _INFINITY)
    printf( (infinite) );

```

```

else
printf(F T, limit.rlim_cur);
if (limit.rlim_max == RLI _INFINITY)
printf( (infinite) );
else
printf(F T, limit.rlim_max);
putchar((int) n);

```

### Unix Kernel Support For Process:

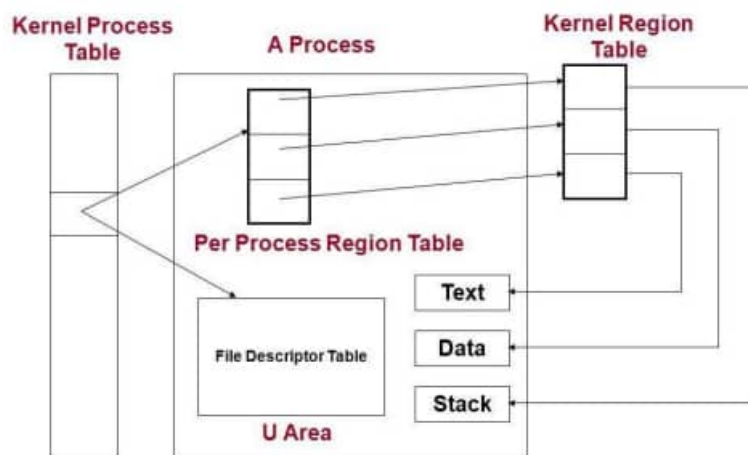
The data structure and execution of processes are dependent on operating system implementation.

A NI process consists minimally of a text segment, a data segment and a stack segment. A segment is an area of memory that is managed by the system as a unit.

- ✓ A text segment consists of the program text in machine executable instruction code format.
- ✓ The data segment contains static and global variables and their corresponding data.
- ✓ A stack segment contains runtime variables and the return addresses of all active functions for a process.

NI kernel has a process table that keeps track of all active process present in the system. Some of these processes belongs to the kernel and are called as “system process”. Every entry in the process table contains pointers to the text, data and the stack segments and also to area of a process. area of a process is an extension of the process table entry and contains other process specific data such as the file descriptor table, current root and working directory inode numbers and set of system imposed process limits.

### Data Structures for Process



All processes in NI system expect the process that is created by the system boot code, are created by the fork system call. After the fork system call, once the child process is created, both the parent and child processes resumes execution. When a process is created by fork, it contains duplicated copies of the text, data and stack segments of its parent as shown in the Figure below. Also it has a file descriptor table, which contains reference to the same opened files as the parent, such that they both share the same file pointer to each opened files.

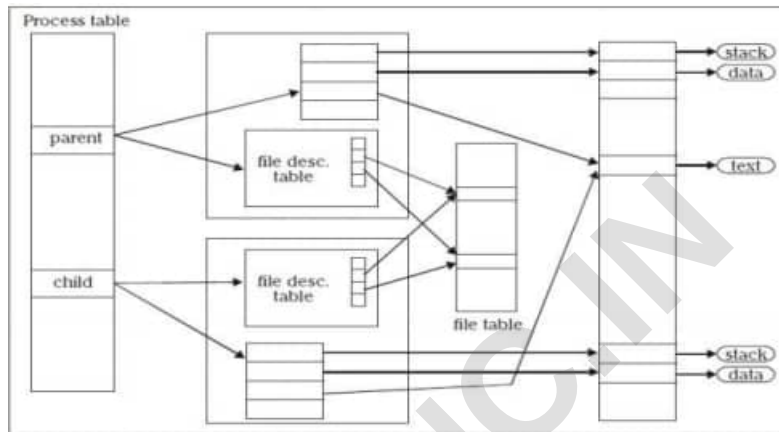


Fig: Parent Child relationship after fork

The process will be assigned with attributes, which are either inherited from its parent or will be set by the kernel.

- ✓ **A real user identification number (rUID):** the user ID of a user who created the parent process.
- ✓ **A real group identification number (rGID):** the group ID of a user who created that parent process.
- ✓ **An effective user identification number (eUID):** this allows the process to access and create files with the same privileges as the program file owner.
- ✓ **An effective group identification number (eGID):** this allows the process to access and create files with the same privileges as the group to which the program file belongs.
- ✓ **Saved set-UID and saved set-GID:** these are the assigned eUID and eGID of the process respectively.
- ✓ **Process group identification number (PGID) and session identification number (SID):** these identify the process group and session of which the process is member.
- ✓ **Supplementary group identification numbers:** this is a set of additional group IDs for a user who created the process.
- ✓ **Current directory:** this is the reference (inode number) to a working directory file.
- ✓ **Root directory:** this is the reference to a root directory.
- ✓ **Signal handling:** the signal handling settings.

- ✓ **Signal mask:** a signal mask that specifies which signals are to be blocked.
- ✓ **Unmask:** a file mode mask that is used in creation of files to specify which accession rights should be taken out.
- ✓ **Nice value:** the process scheduling priority value.
- ✓ **Controlling terminal:** the controlling terminal of the process.

In addition to the above attributes, the following attributes are different between the parent and child processes:

- ✓ **Process identification number (PID):** an integer identification number that is unique per process in an entire operating system.
- ✓ **Parent process identification number (PPID):** the parent process PID.
- ✓ **Pending signals:** the set of signals that are pending delivery to the parent process.
- ✓ **Alarm clock time:** the process alarm clock time is reset to zero in the child process.
- ✓ **File locks:** the set of file locks owned by the parent process is not inherited by the child process.

fork and exec are commonly used together to spawn a sub process to execute a different program. The advantages of this method are:

- ✓ A process can create multiple processes to execute multiple programs concurrently.
- ✓ Because each child process executes in its own virtual address space, the parent process is not affected by the execution status of its child process.

# PROCESS CONTROL

## INTRODUCTION

Process control is concerned about creation of new processes, program execution, and process termination.

## PROCESS IDENTIFIERS

**#include <unistd.h>**

**pid\_t getpid(void);**

Returns: process ID of calling process

**pid\_t getppid(void);**

Returns: parent process ID of calling process

**uid\_t getuid(void);**

Returns: real user ID of calling process

**uid\_t geteuid(void);**

Returns: effective user ID of calling process

**gid\_t getgid(void);**

Returns: real group ID of calling process

**gid\_t getegid(void);**

Returns: effective group ID of calling process

## fork FUNCTION

An existing process can create a new one by calling the forkfunction.

**#include <unistd.h>**

**pid\_t fork(void);**

Returns: 0 in child, process ID of child in parent, 1 on error.

- The new process created by fork is called the child process.
- This function is called once but returns twice.
- The only difference in the returns is that the return value in the child is 0, whereas the return value in the parent is the process ID of the new child.
- The reason the child's process ID is returned to the parent is that a process can have more than one child, and there is no function that allows a process to obtain the process IDs of its children.
- The reason fork returns 0 to the child is that a process can have only a single parent, and the child can always call getppid to obtain the process ID of its parent. (Process ID 0 is reserved for use by the kernel, so it's not possible for 0 to be the process ID of a child.)
- Both the child and the parent continue executing with the instruction that follows the call to fork.
- The child is a copy of the parent.
- For example, the child gets a copy of the parent's data space, heap, and stack.
- Note that this is a copy for the child; the parent and the child do not share these portions of memory.
- The parent and the child share the text segment.

Example programs:

### **Program 1**

/\* Program to demonstrate fork function Program name – fork1.c \*/

```
#include<sys/types.h>
#include<unistd.h> int main(
)
{
    fork( );
    printf("\n hello UP");
}
```

Output :

\$ cc fork1.c

\$ ./a.out

Hello UP

hello UP

Note : The statement hello UP is executed twice as both the child and parent have executed that instruction.

### **Program 2**

/\* Program name – fork2.c \*/

```
#include<sys/types.h>
#include<unistd.h>
int main( )
{
    printf("\n 5th sem ");
    fork( );
    printf("\n hello UP");
}
```

Output :

\$ cc fork1.c

\$ ./a.out

5<sup>th</sup> sem

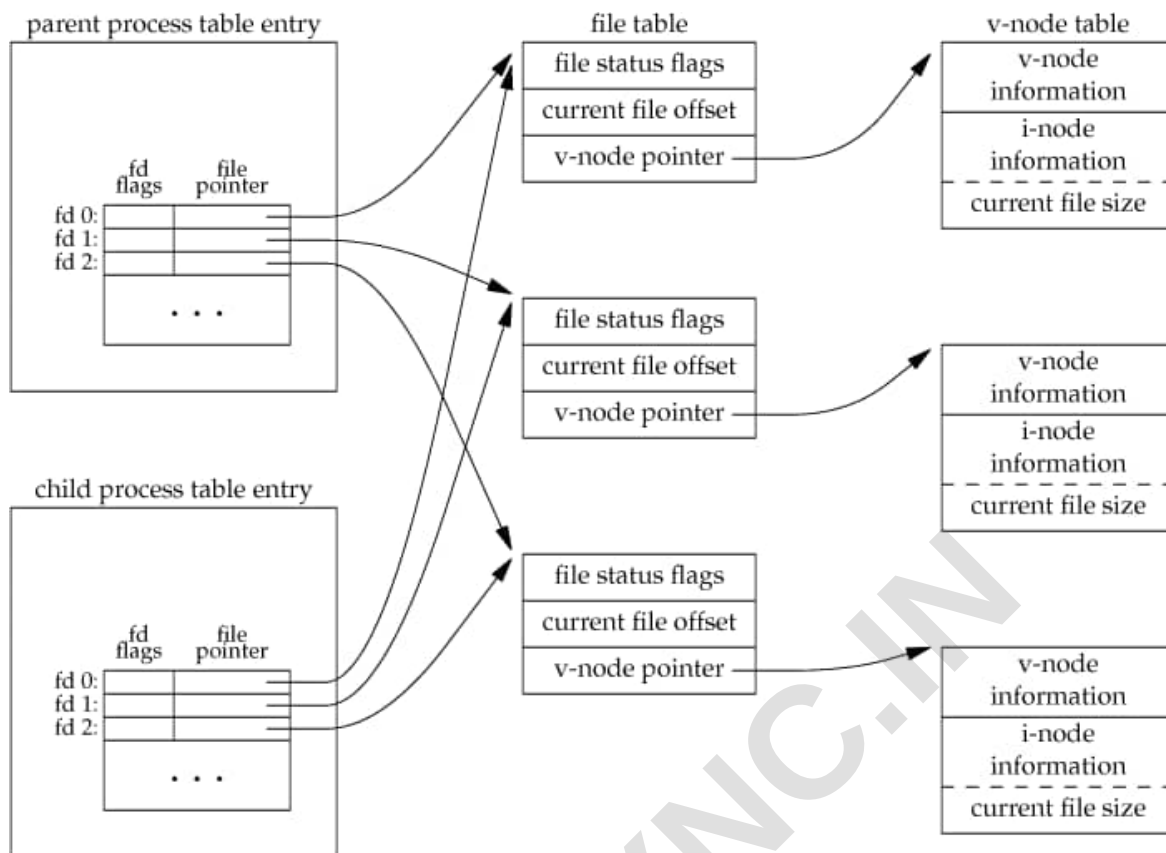
hello UP

hello UP

Note: The statement 5<sup>th</sup> sem is executed only once by the parent because it is called before fork and statement hello UP is executed twice by child and parent. [[\*Also refer lab program 3.sh\*](#)]

### **File Sharing**

Consider a process that has three different files opened for standard input, standard output, and standard error. On return from fork, we have the arrangement shown in Figure 8.2.



**Figure 8.2 Sharing of open files between parent and child after fork**

- It is important that the parent and the child share the same file offset.
- Consider a process that forks a child, then waits for the child to complete.
- Assume that both processes write to standard output as part of their normal processing.
- If the parent has its standard output redirected (by a shell, perhaps) it is essential that the parent's file offset be updated by the child when the child writes to standard output.
- In this case, the child can write to standard output while the parent is waiting for it; on completion of the child, the parent can continue writing to standard output, knowing that its output will be appended to whatever the child wrote.
- If the parent and the child did not share the same file offset, this type of interaction would be more difficult to accomplish and would require explicit actions by the parent.

There are two normal cases for handling the descriptors after a fork.

- ✓ The parent waits for the child to complete. In this case, the parent does not need to do anything with its descriptors. When the child terminates, any of the shared descriptors that the child read from or wrote to will have their file offsets updated accordingly.
- ✓ Both the parent and the child go their own ways. Here, after the fork, the parent closes the descriptors that it doesn't need, and the child does the same thing. This way, neither interferes with the other's open descriptors. This scenario is often the case with network servers.

There are numerous other properties of the parent that are inherited by the child:

- Real user ID, real group ID, effective user ID, effective group ID
- Supplementary group IDs
- Process group ID
- Session ID

- Controlling terminal
- The set-user-ID and set-group-ID flags
- Current working directory
- Root directory
- File mode creation mask
- Signal mask and dispositions
- The close-on-exec flag for any open file descriptors
- Environment
- Attached shared memory segments
- Memory mappings
- Resource limits

The differences between the parent and child are

- ▶ The return value from fork
- ▶ The process IDs are different
- ▶ The two processes have different parent process IDs: the parent process ID of the child is the parent; the parent process ID of the parent doesn't change
- ▶ The child's `tms_utime`, `tms_stime`, `tms_cutime`, and `tms_cstime` values are set to 0
- ▶ File locks set by the parent are not inherited by the child
- ▶ Pending alarms are cleared for the child
- ▶ The set of pending signals for the child is set to the empty set

The two main reasons for `fork` to fail are

- (a) if too many processes are already in the system, which usually means that something else is wrong, or
- (b) if the total number of processes for this real user ID exceeds the system's limit.

There are two uses for `fork`:

- ❖ When a process wants to duplicate itself so that the parent and child can each execute different sections of code at the same time. This is common for network servers, the parent waits for a service request from a client. When the request arrives, the parent calls `fork` and lets the child handle the request. The parent goes back to waiting for the next service request to arrive.
- ❖ When a process wants to execute a different program. This is common for shells. In this case, the child does an `exec` right after it returns from the `fork`.

### **vfork FUNCTION**

- ✓ The function `vfork` has the same calling sequence and same return values as `fork`.
- ✓ The `vfork` function is intended to create a new process when the purpose of the new process is to `exec` a new program.
- ✓ The `vfork` function creates the new process, just like `fork`, without copying the address space of the parent into the child, as the child won't reference that address space; the child simply calls `exec` (or `exit`) right after the `vfork`.
- ✓ Instead, while the child is running and until it calls either `exec` or `exit`, the child runs in the address space of the parent. This optimization provides an efficiency gain on some paged virtual-memory implementations of the UNIX System.

- ✓ Another difference between the two functions is that vfork guarantees that the child runs first, until the child calls exec or exit. When the child calls either of these functions, the parent resumes.

#### Example of vfork function

```
#include "apue.h"
int      glob = 6;          /* external variable in initialized data */

int main(void)
{
    int      var;            /* automatic variable on the stack */ pid_t      pid;

    var = 88;
    printf("before vfork\n");          /* we don't flush stdio */ if ((pid =
    vfork()) < 0) {
        err_sys("vfork error");
    } else if (pid == 0) {              /* child */
        glob++;                        /* modify parent's variables */ var++;
        _exit(0);                      /* child terminates */
    }
    /*
     * Parent continues here.
     */
    printf("pid = %d, glob = %d, var = %d\n", getpid(), glob, var); exit(0);
}
```

#### Output:

```
$ ./a.out before
vfork
pid = 29039, glob = 7, var = 89
```

### exit FUNCTIONS

A process can terminate normally in five ways:

- Executing a return from the main function.
  - Calling the exit function.
  - Calling the \_exit or \_Exit function.
- In most UNIX system implementations, exit(3) is a function in the standard C library, whereas \_exit(2) is a system call.
- Executing a return from the start routine of the last thread in the process. When the last thread returns from its start routine, the process exits with a termination status of 0.
  - Calling the pthread\_exit function from the last thread in the process.

The three forms of abnormal termination are as follows:

- Calling abort. This is a special case of the next item, as it generates the SIGABRT signal.
- When the process receives certain signals. Examples of signals generated by the kernel include the process referencing a memory location not within its address space or trying to divide by 0.
- The last thread responds to a cancellation request. By default, cancellation occurs in a deferred manner: one thread requests that another be canceled, and sometime later, the target thread terminates.

### wait AND waitpid FUNCTIONS

When a process terminates, either normally or abnormally, the kernel notifies the parent by sending the SIGCHLD signal to the parent. Because the termination of a child is an asynchronous event - it can happen at any time while the parent is running - this signal is the asynchronous notification from the kernel to the parent. The parent can choose to ignore this signal, or it can provide a function that is called when the signal occurs: a signal handler.

A process that calls wait or waitpid can:

- ✓ Block, if all of its children are still running
- ✓ Return immediately with the termination status of a child, if a child has terminated and is waiting for its termination status to be fetched
- ✓ Return immediately with an error, if it doesn't have any child processes.

```
#include <sys/wait.h> pid_t
```

```
wait(int *statloc);
```

```
pid_t waitpid(pid_t pid, int *statloc, int options);
```

Both return: process ID if OK, 0 (see later), or 1 on error.

The differences between these two functions are as follows.

- ▶ The wait function can block the caller until a child process terminates, whereas waitpid has an option that prevents it from blocking.
- ▶ The waitpid function doesn't wait for the child that terminates first; it has a number of options that control which process it waits for.

If a child has already terminated and is a zombie, wait returns immediately with that child's status. Otherwise, it blocks the caller until a child terminates. If the caller blocks and has multiple children, wait returns when one terminates.

For both functions, the argument statloc is a pointer to an integer. If this argument is not a null pointer, the termination status of the terminated process is stored in the location pointed to by the argument.

Print a description of the exit status

```
#include "apue.h" #include
<sys/wait.h>
```

```
Void pr_exit(int status)
{
    if (WIFEXITED(status))
        printf("normal termination, exit status = %d\n",
               WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("abnormal termination, signal number = %d%s\n",
               WTERMSIG(status),
#ifdef WCOREDUMP
               WCOREDUMP(status) ? " (core file generated)" : "";
#else
               "");
#endif

    else if (WIFSTOPPED(status))
        printf("child stopped, signal number = %d\n",
               WSTOPSIG(status));
}
```

Program to Demonstrate various exit statuses

```
#include "apue.h" #include
<sys/wait.h>
```

```
Int main(void)
{
    pid_t    pid;
    int      status;

    if ((pid = fork()) < 0) err_sys("fork
        error");
    else if (pid == 0)                /* child */
        exit(7);

    if (wait(&status) != pid)        /* wait for child */
        err_sys("wait error");
    pr_exit(status);                 /* and print its status */
}
```

```

if ((pid = fork()) < 0) err_sys("fork
    error");
else if (pid == 0)                /* child */
    abort();                      /* generates SIGABRT */

if (wait(&status) != pid)        /* wait for child */
    err_sys("wait error");
pr_exit(status);                /* and print its status */

if ((pid = fork()) < 0) err_sys("fork
    error");
else if (pid == 0)                /* child */
    status /= 0;                 /* divide by 0 generates SIGFPE */

if (wait(&status) != pid)        /* wait for child */
    err_sys("wait error");
pr_exit(status);                /* and print its status */

exit(0);
}

```

The interpretation of the pid argument for waitpid depends on its value:

|          |                                                                                |
|----------|--------------------------------------------------------------------------------|
| pid == 1 | Waits for any child process. In this respect, waitpid is equivalent to wait.   |
| pid > 0  | Waits for the child whose process ID equals pid.                               |
| pid == 0 | Waits for any child whose process group ID equals that of the calling process. |
| pid < 1  | Waits for any child whose process group ID equals the absolute value of pid.   |

#### Macros to examine the termination status returned by wait and waitpid

| Macro                | Description                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WIFEXITED(status)    | True if status was returned for a child that terminated normally. In this case, we can execute WEXITSTATUS(status) to fetch the low-order 8 bits of the argument that the child passed to exit, _exit, or _Exit.                                                                                                                                                                                               |
| WIFSIGNALED(status)  | True if status was returned for a child that terminated abnormally, by receipt of a signal that it didn't catch. In this case, we can execute WTERMSIG(status) to fetch the signal number that caused the termination. Additionally, some implementations (but not the Single UNIX Specification) define the macro WCOREDUMP(status) that returns true if a core file of the terminated process was generated. |
| WIFSTOPPED(status)   | True if status was returned for a child that is currently stopped. In this case, we can execute WSTOPSIG(status) to fetch the signal number that caused the child to stop.                                                                                                                                                                                                                                     |
| WIFCONTINUED(status) | True if status was returned for a child that has been continued after a job control stop.                                                                                                                                                                                                                                                                                                                      |

| The options constants for waitpid |                                                                                                                                                                                                                                                                                   |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Constant                          | Description                                                                                                                                                                                                                                                                       |
| <b>WCONTINUED</b>                 | If the implementation supports job control, the status of any child specified by pid that has been continued after being stopped, but whose status has not yet been reported, is returned.                                                                                        |
| <b>WNOHANG</b>                    | The waitpid function will not block if a child specified by pid is not immediately available. In this case, the return value is 0.                                                                                                                                                |
| <b>WUNTRACED</b>                  | If the implementation supports job control, the status of any child specified by pid that has stopped, and whose status has not been reported since it has stopped, is returned. The WIFSTOPPED macro determines whether the return value corresponds to a stopped child process. |

The waitpid function provides three features that aren't provided by the wait function.

- ✓ The waitpid function lets us wait for one particular process, whereas the wait function returns the status of any terminated child. We'll return to this feature when we discuss the popen function.
- ✓ The waitpid function provides a nonblocking version of wait. There are times when we want to fetch a child's status, but we don't want to block.
- ✓ The waitpid function provides support for job control with the WUNTRACED and WCONTINUED options.

Program to Avoid zombie processes by calling fork twice

```
#include "apue.h" #include
<sys/wait.h>
```

```
Int main(void)
{
    pid_t    pid;

    if ((pid = fork()) < 0) { err_sys("fork
        error");
    } else if (pid == 0) { /* first child */ if ((pid =
        fork()) < 0)
        err_sys("fork error"); else if
        (pid > 0)
            exit(0); /* parent from second fork == first child */
        /*
        * We're the second child; our parent becomes init as soon
        * as our real parent calls exit() in the statement above.
        * Here's where we'd continue executing, knowing that when
        * we're done, init will reap our status.
        */ sleep(2);
        printf("second child, parent pid = %d\n", getppid()); exit(0);
    }

    if (waitpid(pid, NULL, 0) != pid) /* wait for first child */
        err_sys("waitpid error");

    /*
    * We're the parent (the original process); we continue executing,
    * knowing that we're not the parent of the second child.
    */ exit(0);
}
```

**Output:**

```
$ ./a.out
$ second child, parent pid = 1
```

### waitid FUNCTION

The waitid function is similar to waitpid, but provides extra flexibility.

```
#include <sys/wait.h>
```

```
int waitid(idtype_t idtype, id_t id, siginfo_t *infop, int options);
```

Returns: 0 if OK, -1 on error

The *idtype* constants for waited are as follows:

| Constant      | Description                                                                                                             |
|---------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>P_PID</b>  | Wait for a particular process: id contains the process ID of the child to wait for.                                     |
| <b>P_PGID</b> | Wait for any child process in a particular process group: id contains the process group ID of the children to wait for. |
| <b>P_ALL</b>  | Wait for any child process: id is ignored.                                                                              |

The options argument is a bitwise OR of the flags as shown below: these flags indicate which state changes the caller is interested in.

| Constant         | Description                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>WCONTINUE</b> | Wait for a process that has previously stopped and has been continued, and whose status has not yet been reported.              |
| <b>WEXITED</b>   | Wait for processes that have exited.                                                                                            |
| <b>WNOHANG</b>   | Return immediately instead of blocking if there is no child exit status available.                                              |
| <b>WNOWAIT</b>   | Don't destroy the child exit status. The child's exit status can be retrieved by a subsequent call to wait, waitid, or waitpid. |
| <b>WSTOPPED</b>  | Wait for a process that has stopped and whose status has not yet been reported.                                                 |

### wait3 AND wait4 FUNCTIONS

The only feature provided by these two functions that isn't provided by the wait, waitid, and waitpid functions is an additional argument that allows the kernel to return a summary of the resources used by the terminated process and all its child processes.

The prototypes of these functions are:

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <sys/time.h>
```

```
#include <sys/resource.h>
```

```
pid_t wait3(int *statloc, int options, struct rusage *rusage);
```

```
pid_t wait4(pid_t pid, int *statloc, int options, struct rusage *rusage);
```

Both return: process ID if OK, -1 on error

The resource information includes such statistics as the amount of user CPU time, the amount of system CPU time, number of page faults, number of signals received etc. the resource information is available only for terminated child process not for the process that were stopped due to job control.

### RACE CONDITIONS

A race condition occurs when multiple processes are trying to do something with shared data and the final outcome depends on the order in which the processes run.

**Example:** The program below outputs two strings: one from the child and one from the parent. The program contains a race condition because the output depends on the order in which the processes are run by the kernel and for how long each process runs.

```
#include "apue.h"

static void charatime(char *); int

main(void)
{
    pid_t    pid;

    if ((pid = fork()) < 0) { err_sys("fork
        error");
    } else if (pid == 0) { charatime("output from
        child\n");
    } else {
        charatime("output from parent\n");
    }
    exit(0);
}

static void charatime(char
*str)
{
    char      *ptr;
    int       c;

    setbuf(stdout, NULL);          /* set unbuffered */ for (ptr
    = str; (c = *ptr++) != 0; )
        putc(c, stdout);
}
```

#### Output:

```
$ ./a.out
output from child utput
from parent
$ ./a.out
output from child utput
from parent
$ ./a.out
output from child output
from parent
```

#### program modification to avoid race condition

```
#include "apue.h"

static void charatime(char *); int

main(void)
{
    pid_t    pid;

    TELL_WAIT();

    if ((pid = fork()) < 0) { err_sys("fork
        error");
    } else if (pid == 0) {
        WAIT_PARENT();          /* parent goes first */
        charatime("output from child\n");
    } else {
        charatime("output from parent\n");
        TELL_CHILD(pid);
    }
    exit(0);
}

static void
```

```

charatotime(char *str)
{
    char    *ptr;
    int     c;

    setbuf(stdout, NULL);          /* set unbuffered */ for (ptr
    = str; (c = *ptr++) != 0; )
        putc(c, stdout);
}

```

When we run this program, the output is as we expect; there is no intermixing of output from the two processes.

## execFUNCTIONS

When a process calls one of the exec functions, that process is completely replaced by the new program, and the new program starts executing at its main function. The process ID does not change across an exec, because a new process is not created; exec merely replaces the current process - its text, data, heap, and stack segments - with a brand new program from disk.

There are 6 exec functions:

```

#include <unistd.h>
int  execl(const char *pathname, const char *arg0, ... /* (char *)0 */ );
int  execv(const char *pathname, char *const argv []);
int  execlp(const char *pathname, const char *arg0, ... /*(char *)0, char
*const envp */ );
int  execve(const char *pathname, char *const argv[], char *const envp[]);
int  execlp(const char *filename, const char *arg0, ... /* (char *)0 */ );
int  execvp(const char *filename, char *const argv []);

```

All six return: -1 on error, no return on success.

- ❖ The first difference in these functions is that the first four take a pathname argument, whereas the last two take a filename argument. When a filename argument is specified
  - If filename contains a slash, it is taken as a pathname.
  - Otherwise, the executable file is searched for in the directories specified by the PATH environment variable.
- ❖ The next difference concerns the passing of the argument list (l stands for list and v stands for vector). The functions execl, execlp, and execlerequire each of the command-line arguments to the new program to be specified as separate arguments. For the other three functions (execv, execvp, and execve), we have to build an array of pointers to the arguments, and the address of this array is the argument to these three functions.
- ❖ The final difference is the passing of the environment list to the new program. The two functions whose names end in an e (execl and execve) allow us to pass a pointer to an array of pointers to the environment strings. The other four functions, however, use the environ variable in the calling process to copy the existing environment for the new program.

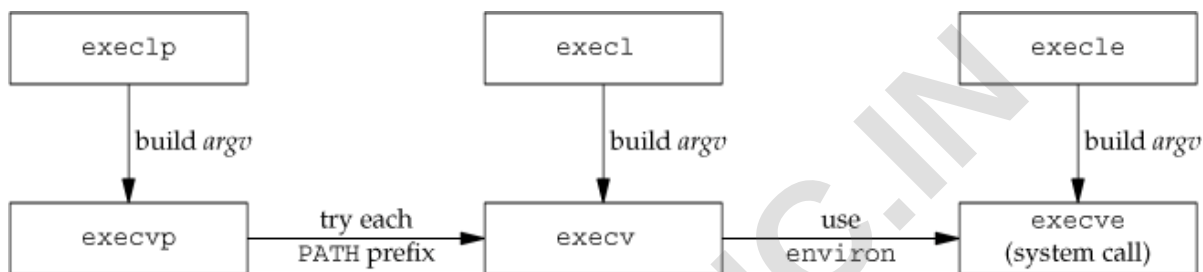
| Function         | pathname | filename | Arg list | argv[] | environ | envp[] |
|------------------|----------|----------|----------|--------|---------|--------|
| execl            | •        |          | •        |        | •       |        |
| execlp           |          | •        | •        |        | •       |        |
| execl            | •        |          | •        |        |         | •      |
| execv            | •        |          |          | •      | •       |        |
| execvp           |          | •        |          | •      | •       |        |
| execve           | •        |          |          | •      |         | •      |
| (letter in name) |          | p        | l        | v      |         | e      |

The above table shows the differences among the 6 exec functions.

We've mentioned that the process ID does not change after an exec, but the new program inherits additional properties from the calling process:

- Process ID and parent process ID

- Real user ID and real group ID
- Supplementary group IDs
- Process group ID
- Session ID
- Controlling terminal
- Time left until alarm clock
- Current working directory
- Root directory
- File mode creation mask
- File locks
- Process signal mask
- Pending signals
- Resource limits
- Values for tms\_utime, tms\_stime, tms\_cutime, and tms\_cstime.



**Relationship of the six exec functions**

**Example of execfunctions**

```

#include "apue.h"
#include <sys/wait.h>

char    *env_init[] = { "USER=unknown", "PATH=/tmp", NULL };

int main(void)
{
    pid_t    pid;

    if ((pid = fork()) < 0) { err_sys("fork
        error");
    } else if (pid == 0) {
        /* specify pathname, specify environment */ if
        (execl("/home/sar/bin/echoall", "echoall", "myarg1",
            "MY ARG2", (char *)0, env_init) < 0)
            err_sys("execl error");
    }

    if (waitpid(pid, NULL, 0) < 0)
        err_sys("wait error");

    if ((pid = fork()) < 0) { err_sys("fork error");
    } else if (pid == 0) {
        /* specify filename, inherit environment */ if
        (execlp("echoall", "echoall", "only 1 arg", (char *)0) < 0)
            err_sys("execlp error");
    }

    exit(0);
}
  
```

**Output:**

```

$ ./a.out
argv[0]: echoall
argv[1]: myarg1
argv[2]: MY ARG2
USER=unknown
  
```

```
PATH=/tmp
$ argv[0]: echoall argv[1]:
only 1 arg USER=sar
LOGNAME=sar
SHELL=/bin/bash
```

```
HOME=/home/sar
```

47 more lines that aren't shown

Note that the shell prompt appeared before the printing of argv[0] from the second exec. This is because the parent did not wait for this child process to finish.

Echo all command-line arguments and all environment strings

```
#include "apue.h"
```

```
int main(int argc, char *argv[])
{
    int            i;
    char           **ptr; extern
    char **environ;

    for (i = 0; i < argc; i++)           /* echo all command-line args */
        printf("argv[%d]: %s\n", i, argv[i]);

    for (ptr = environ; *ptr != 0; ptr++) /* and all env strings */
        printf("%s\n", *ptr);

    exit(0);
}
```